

ALGORITMOS Y ESTRUCTURAS DE DATOS AVANZADOS

MEMORIA PRACTICA 1

Enrique Gómez Fernández

Daniel Birsan

Grupo 10

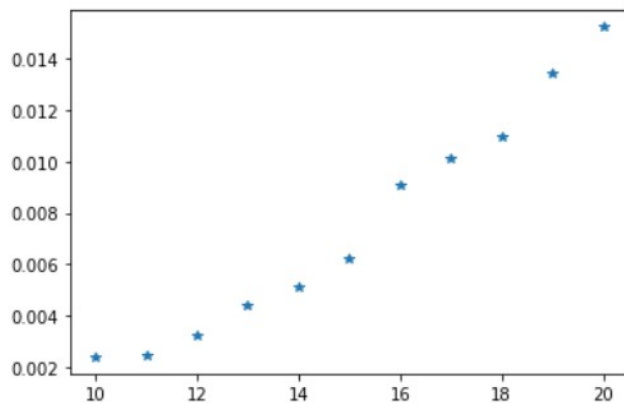
I-C. Cuestiones-Python Básico

Ejercicio 1

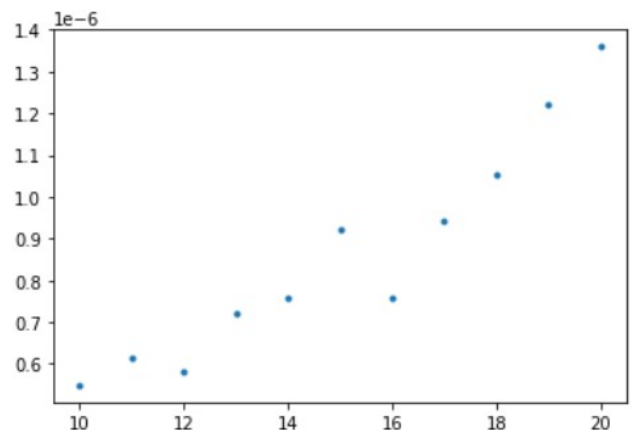
Al ser las matrices generadas cuadradas y de la misma dimensión, con N filas y columnas, f se debería ajustar a la función $(N*N*N)=N^3$.

Ejercicio 2

Con nuestra función:



Con a.dot:

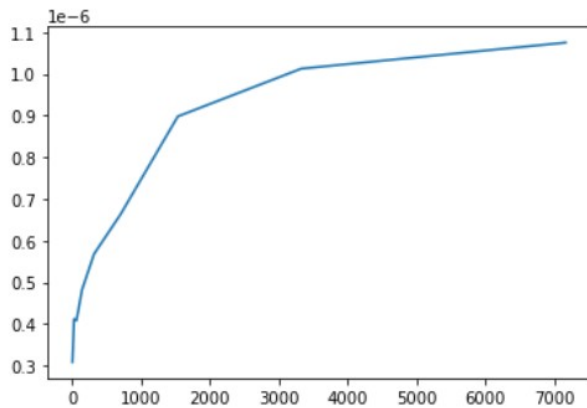


Como se puede apreciar, la función que siguen es parecida, sin embargo el tiempo requerido con cada valor de N es 10^3 veces mayor con la función `matrix_multiplication` que usando `a.dot`.

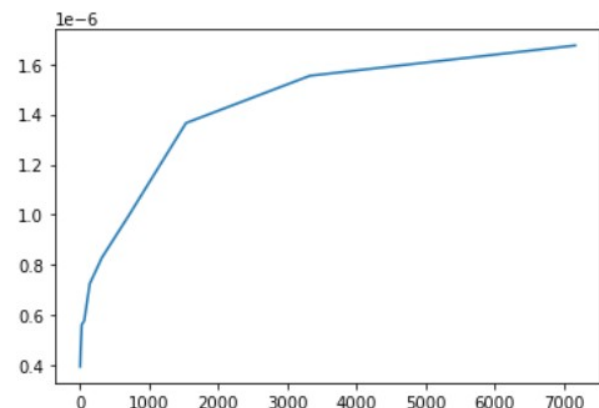
Ejercicio 3:

Para este ejercicio hemos medido los tiempos de ejecución de una búsqueda binaria realizada de forma recursiva e iterativa. Esto lo hemos medido con el caso en sus casos mas costosos, es decir, buscando un numero que no estuviera en la lista utilizando (size + 1).

Iterativa:



Recursiva:



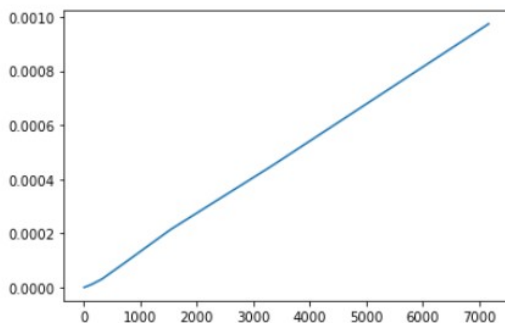
Como se puede apreciar, la forma que hace la función de ambas es muy parecida en cuanto a forma. Sin embargo, la recursiva es mas costosa en cuanto a tiempo ya que necesita llamarse a si misma lo que conlleva un mayor gasto de tiempo. Sin embargo ambas se asemejan a la función $f(n)=\log(n)$.

II-D. Cuestiones

Ejercicio 1

La función `create_min_heap` tiene 2 bucles anidados. El primero recorre todos los nodos que no son hojas para posteriormente hacer un `heapify` desde ese nodo. Para N igual al numero de números que posee la array inicial, la función de `heapify` se debe ajustar a $\log_2(N)$ mientras que el numero de nodos que se recorrerán es $N/2$. Por tanto esta ecuación se ajusta a la de $f(N) = N \log_2(N)/2$.

Función de tiempo de ejecución de creación de min heaps:



Podemos apreciar obviamente una función creciente que se asemeja en gran medida a la expuesta anteriormente $N \log_2(N)/2$. Además es lógica ya que para $N=0$ el tiempo es 0 ya que no se crea la array y este sube a medida que N es mayor.

Ejercicio 2

De primeras, recorrer el array entero, $N = \text{len}(\text{array})$, además, la comparación de los elementos requiere 1 única operación (N). Ahora bien, esta comparación puede ser exitosa o no, por tanto, $O(k) = O(\log n)$ para una comparación con éxito y N para una sin éxito. Con esto concluimos que la eficiencia del algoritmo es $1 \leq O(k) = O(\log n) \leq N$ si tiene éxito y N si no lo tiene.

Ejercicio 3

Nuestro algoritmo proporciona los k primeros elementos ordenados del array puesto que tenemos que crear un max heap para la selección del k ésimo elemento, si en el heap únicamente insertamos los elementos menores que el padre del heap, eliminando el padre, y por cada inserción aplicamos `heapify` de tal forma que el padre siempre sea el mayor, cuando se recorra todo el array los k elementos del heap serán los primeros k elementos ordenados del array. Además, nos da los k primeros debido a que el heap solo tendrá profundidad k para poder aplicar el algoritmo de selección.

Ejercicio 4

Con nuestro algoritmo se pueden obtener estos dos elementos porque como hemos visto en la cuestión anterior también nos da los primeros k elementos del array, por tanto, al finalizar el recorrido del array podemos extraer el padre del heap, este será el k ésimo elemento, pero si nuevamente extraemos el padre, este será el $k-1$ ésimo elemento que es justo lo que nos piden.

El problema llega cuando $k = 1$, es decir, el elemento más pequeño del array. En este caso nuestro algoritmo solo devolvería el k ésimo elemento, y para conseguir el $k-1$ elemento deberíamos volver a repetir el algoritmo, entonces esta vez utilizaríamos dos bucles for.